TUGAS I - IF5180 INTELIGENSI BUATAN LANJUT

ARIEF PURNAMA MUHARRAM - 23521013

RESUME SINGKAT PENDEKATAN CSP DAN PENJELASAN SINGKAT ALGORITMA *BACKTRACKING* DENGAN MAC

Constraint Satisfaction Problem (CSP) merupakan suatu permasalahan yang melibatkan suatu nilai *(value)*, yang dipilih dari suatu domain terbatas *(finite domain)*, untuk dimasukan *(assigned)* ke dalam variabel dalam suatu kasus permasalahan, sehingga seluruh batasan-batasan *(constraint)* yang berhubungan dengan variabel tersebut terpenuhi. Salah satu contoh nyata *(real-world problem)* dari CSP yaitu terkait dengan *scheduling* dan *timetabling*.

Problem Definition

Suatu CSP terdiri atas komponen-komponen sebagai berikut.

- Sebuah set ***variabel*** $X = \{x_1, \ldots, x_n\}$
- Untuk setiap variabel x_i, terdapat suatu **domain** D_i berisikan nilai-nilai variabel yang memungkinkan.
- Sebuah set ***constraint*** yang membatasi nilai-nilai yang dapat diambil untuk setiap variabel.

Suatu solusi CSP dapat tercapai apabila nilai-nilai yang dimasukan ke dalam variabel dari domain yang didefinisikan memenuhi *constraint* yang diberikan. Disisi lain, jika tidak terdapat nilai variabel dari domain yang dapat memenuhi *constraint* yang diberikan maka solusi terhadap masalah tersebut tidak dapat tercapai *(unsatisfiable)*.

Melalui pendekatan CSP maka diharapkan dapat menemukan solusi sebagai berikut.

- Cukup satu solusi, tanpa preferensi terhadap solusi lain.
- Seluruh solusi yang memungkinkan.
- Sebuah solusi optimal, atau setidaknya, solusi terbaik.

Constraint

Suatu *constraint* pada CSP umumnya direpresentasikan oleh suatu ekspresi matematika yang memengaruhi variabel. Contoh *constraint* seperti $x_1 \neq x_2, 2x_1 = 10x_2 + x_3$, dan $x_1 x_2 < x_3$.

Secara formal, suatu *constraint* $C_{ijk\ldots}$ antara variabel-variabel x_i,x_j,x_k,... merupakan *subset* dari setiap kombinasi yang memungkinkan dari nilai-nilai x_i,x_j,x_k,... sehingga dapat direpresentasikan sebagai $C_{ijk\ldots} \subseteq D_i \times D_j \times D_k \times \ldots$

Sebagai contoh, jika suatu variabel x memiliki domain $\{1,2,3\}$ dan variabel y memiliki domain $\{1,2\}$ maka kombinasi nilai yang memungkinkan adalah $\{(1,1), (1,2), (2,1), (2,2), (3,1), (3,2)\}$. Apabila diberikan suatu *constraint* $x = y$ maka *subset* dari *constraint* yang dihasilkan adalah $\{(1,1), (2,2)\}$.

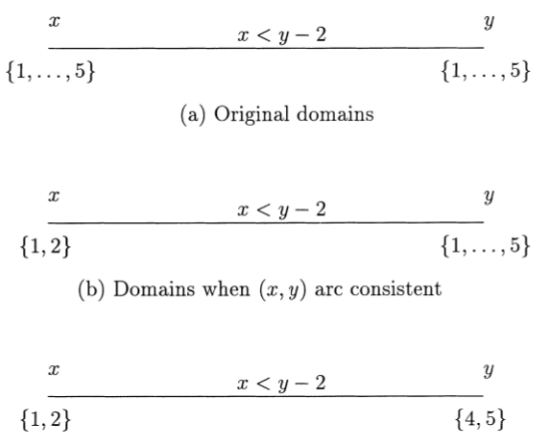
Arc Consistency

Jika seluruh *constraint* dari CSP merupakan bentuk biner maka variabel dan *constraint* tersebut dapat direpresentasikan dalam suatu graf *(graph)*. Pada graf yang merepresentasikan suatu CSP, *node* merepresentasikan variabel, sedangkan *edge* merepresentasikan *constraint* diantara kedua variabel.

Jika terdapat suatu *binary constraint* C_{ij}antara variabel x_i dan x_j, kemudian suatu *directed arc* (x_i, x_j) akan konsisten apabila untuk setiap nilai $a \in D_i$, selalu terdapat nilai $b \in D_j$ sehingga nilai $x_i = a$ dan $x_j = b$ memenuhi *constraint* C_{ij}.

Setiap nilai $a \in D_i$ yang bernilai tidak benar *(not true)*, semisalkan tidak terdapat nilai b yang memenuhi, maka dapat dihilangkan dari D_i dikarenakan bukan lagi bagian dari solusi konsisten. Menghilangkan seluruh nilai-nilai yang demikian dapat membuat *arc* (x_i, x_j) menjadi **konsisten**.

Jika setiap *arc* pada suatu *binary CSP* dibuat menjadi konsisten maka keseluruhannya tersebut disebut sebagai ***arc consistent***. Pada beberapa kasus yang sederhana, hasil yang diharapkan setelah membuat seluruh *arc* menjadi konsisten adalah menyisakan nilai-nilai yang tersisa di dalam domain sebagai *feasible solution* dari CSP.



Gambar 1. Membuat *arc consistency*; (a) domain awal; (b) domain ketika (x, y) konsisten; (c) domain ketika (x, y) dan (y, x) konsisten.

Search Algorithms

Dalam pencarian suatu *feasible solution* pada kasus CSP maka diperlukan suatu *search algorithm*. Terdapat tiga *search algortihm* yang dapat digunakan dalam pendekatan penyelesaian permasalahan CSP, yaitu (1) *backtracking*; (2) *forward checking*; dan (3) MAC *(Maintaining Arc Consistency)*.

Seluruh *search algorithm* tersebut melibatkan pembentukan *tree* untuk merepresentasikan *current state* dari proses pencarian. Setiap *node* dari *tree* yang dibentuk berkorespondensi dengan solusi parsial dimana nilai variabel tersebut telah ditentukan; disebut juga sebagai *past variable*. Nilai *future variable* merupakan nilai tersisa untuk ditentukan selanjutnya. Suatu kebuntuan *(deadend)* akan dideteksi apabila nilai domain dari *future variable* menjadi tidak ada *(empty)*. Perbedaan antar ketiga *search algorithm* tersebut terletak pada perlakuannya terhadap ***future variable***.

Search Algorithm: Backtracking

Pada algoritma *backtracking*, nilai variabel saat ini diperoleh melalui domainnya. Selanjutnya, nilai tersebut dibandingkan terhadap solusi parsialnya; jika terdapat *constraint* antara variabel tersebut dengan *past variable* yang dilanggar maka nilai tersebut akan diabaikan dan nilai vairabel lain akan dipilih dari domain yang tersedia. Jika seluruh nilai pada variabel saat ini *(current variable)* telah dicoba maka algoritma akan melakukan *backtrack* ke *past variable* dan mencoba nilai baru yang tersedia. Jika seluruh solusi lengkap ditemukan, misalkan setiap nilai telah dimasukan ke dalam variabel, maka algoritma hanya akan berhenti jika hanya satu solusi yang dibutuhkan, atau mulai melanjutkan proses untuk mencari solusi baru jika dibutuhkan. Jika tidak ditemukan solusi yang sesuai, algoritma akan berhenti dengan kemungkinan solusi-solusi yang dapat diberikan.

Search Algorithm: Forward Checking

Forward checking merupakan pengembangan lebih lanjut dari *backtracking*. Alih-alih hanya

memeriksan *current variable* dan *past variable*, algoritma *forward checking* juga memeriksa *future variable*.

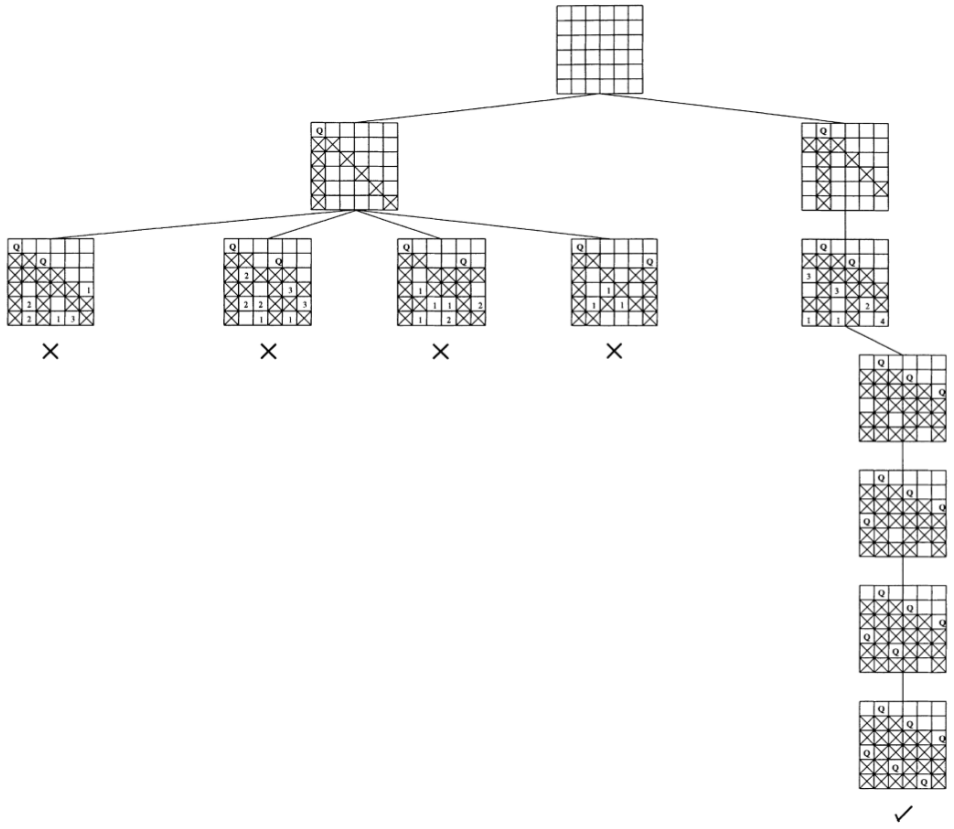
Pada algoritma *forward checking*, ketika suatu nilai dimasukan *(assigned)* ke dalam *current variable*, apabila terdapat nilai dari domain *future variable* yang berkonflik maka akan dihilangkan dari domain.

Keuntungan dari pendekatan ini adalah apabila domain dari *future variable* menjadi kosong *(empty)* maka segera diketahui bahwa solusi parsial tidak konsisten *(inconsistent)* sehingga nilai lain dapat segera dicoba. Pendekatan *forward tracking* memungkinkan *tree* yang terbentuk dipangkas lebih awal daripada dengan pendekatan *backtracking*.

Search Algorithm: MAC

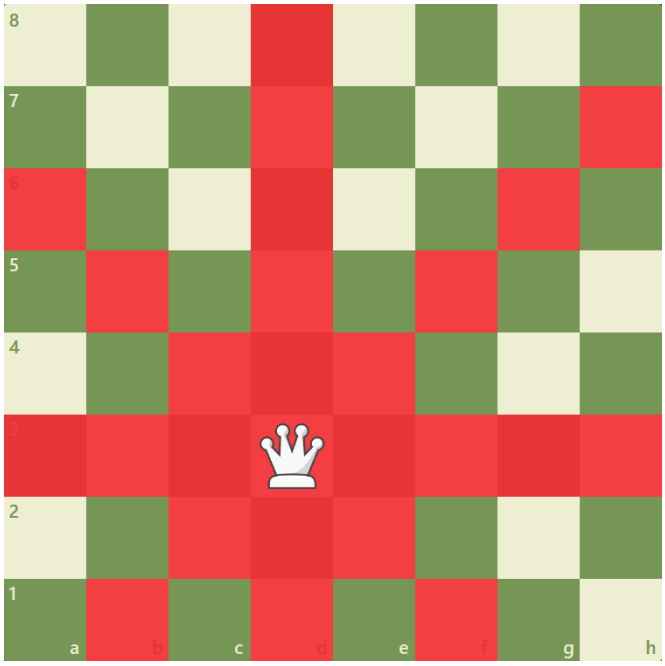
Algoritma MAC merupakan pengembangan lebih lanjut dari *forward tracking*. Pada algoritma MAC, algoritma memeriksa *future variable* terhadap satu sama lain. Dengan demikian, untuk setiap *future variable*, setiap nilai yang tidak didukung pada domain dari *future variable* lain akan dihilangkan. Dengan demikian, pendekatan MAC lebih menghemat waktu komputasi.

Penjelasan Algoritma *Backtracking* dengan MAC



Gambar 2. Contoh implementasi algoritma MAC pada kasus *6-queens problem*

Diketahui sebuah kasus *6-queens problem*. Permasalahan tersebut dapat dituliskan sebagai n variabel $v_1, \ldots, v_n$ yang merepresentasikan baris *(row)* dan setiap variabel memiliki domain $\{1, \ldots, n\}$ yang merepresentasikan kolom *(column)* tempat *queen* tersebut dapat diletakan. Algoritma bekerja dengan memeriksa nilai-nilai yang memungkinkan dari satu baris ke baris selanjutnya. Untuk memudahkan ilustrasi, berikut merupakan gambaran pergerakan bidak *queen* pada permainan catur.



Gambar 3. Pergerakan *queen* pada permainan catur

Sebagai contoh, pada kombinasi penempatan *queen* yang paling kiri. Pada penempatan *queen* di baris pertama (v_1) dicoba diletakkan pada kolom pertama (sehingga $v_1 = 1$). Penempatan *queen* di baris pertama pada kolom pertama mengakibatkan domain yang memungkinkan pada baris kedua (v_2) yaitu $\{3,4,5,6\}$.

Pada percobaan kombinasi pertama, *queen* kedua diletakkan pada baris kedua di kolom ketiga (sehingga $v_2 = 3$). Asumsikan pada posisi ini sebagai *current variable*. Pada penempatan bidak *queen* kedua di $v_2 = 3$ maka nilai domain yang memungkinkan pada baris ketiga sebagai *future variable* adalah $\{5,6\}$. Akan tetapi, perhatikan karena nilai 6 pada baris keempat (v_4) tidak cocok dengan kedua nilai tersebut maka dapat dihilangkan. Penomoran pada **gambar 2** menandakan kemungkinan urutan di mana hal ini dapat dilakukan, karena setelah beberapa nilai dihilangkan, ini dapat memicu penghapusan lebih lanjut.

Gambar 2 paling kanan menunjukkan satu solusi yang mungkin dapat dihasilkan. Dengan demikian, algoritma MAC mengurangi *search space* yang mungkin dilakukan. Apabila diketahui terdapat *arc inconsistency* maka algoritma dapat segera melakukan *backtracking* alih-alih mengakibatkan *search space* dan kemungkinan kandidat yang begitu banyak.